```
enum = "cat".each_char
enum.to_a # => ["c", "a", "t"]
```

If the method we're using as the basis of our enumerator takes parameters, we can pass them to enum_for:

```
enum_in_threes = (1..10).enum_for(:each_slice, 3)
enum_in_threes.to_a # => [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10]]
```

### Enumerators Used as Generators and Filters

In addition to creating enumerators from existing collections, you can create an enumerator explicitly with Enumerator.new, passing it a block that takes a single argument. The code in the block will be used when next is called on the enumerator object and it needs to supply a fresh value to your program. However, the block isn't simply executed from top to bottom. When first called, execution starts at the top of the block and pauses when the block calls yield on its argument, which yields a value to the calling code. When next is called again, execution resumes at the statement following the yield.

Among other things, this lets you write enumerators that generate infinite sequences:

```
triangular_numbers = Enumerator.new do |yielder|
  number = 0
  count = 1
  loop do
    number += count
    count += 1
    yielder.yield(number)
  end
end

5.times { print triangular_numbers.next, " " }
puts
```

*produces:*

```
1 3 6 10 15
```

We start by creating an iterator using Enumerator.new and assigning that value to the variable triangular_numbers. After that, we loop 5.times, each time calling triangular_numbers.next. The first time next is called, we start at the top of the block, setting number and count before entering the loop. In the loop number is set to 1 and count is incremented to 2 and then yielder.yield is called, passing number back to the caller.

The following time that next is called, we continue at the point of yield meaning that we stay in the loop rather than start at the beginning again. The code updates number to 3, increments count to three, and yields. And so on…

That syntax for infinite sequences is confusing, though. So, a simpler mechanism was added for creating infinite sequences. The produce method takes an initial value and a block. Every time the block is invoked via next, the resulting value is stored and used as the input to the next call. This means all you need to do is define the succession function, and you don't need to worry about managing yielders and whatnot:

```
triangular_numbers = Enumerator.produce([1, 2]) do |number, count|
  [number + count, count + 1]
end
```